

AI Assistant MVP - Technical Specification & Architecture Brief

Project Goal

Replace the existing Copilot system with a conversational AI Assistant that combines:

1. Existing Help/Documentation system
2. Existing Copilot workflow framework
3. Live application context awareness
4. Guided user onboarding and workflow assistance

The AI Assistant must become the long-term foundation for a future cross-platform assistant ecosystem, including a dedicated mobile application with access to user schedules, tasks, account information, notifications, and limited write permissions.

This MVP is NOT a standalone chatbot.

This MVP is the first version of a much larger account-aware assistant platform.

High Level Vision

Current State:

- Help system = static documentation
- Copilot = rigid guided tutorial
- Both operate independently

Target State:

- Single conversational assistant
- Floating chat interface
- Natural language interaction
- Context-aware responses
- Documentation-aware responses
- Workflow-aware responses
- Page-aware responses

The assistant should feel like an experienced product expert helping the user navigate the application.

Example:

User: "How do I create a component?"

Assistant: "You're currently on the Components page. Click the Add Component button in the top right to begin."

User: "Where is it?"

Assistant: "It is located above the component table on the right side. I've highlighted it for you."

Assistant triggers UI highlight.

User clicks button.

Assistant automatically detects progression.

Assistant: "Great. Now enter a component name and select a measurement type."

Core Design Principle

The AI should NOT invent workflows.

The application remains the source of truth.

The assistant's job is to:

- Explain
- Guide
- Clarify
- Teach
- Answer questions

The application's job is to:

- Determine workflow state
- Determine current page
- Determine current item
- Determine completion status
- Determine next valid step

The AI explains the next step but does not decide it.

MVP Scope

The MVP assistant must support:

Documentation Assistance

User can ask:

- What does this feature do?
- How do I create a component?
- What is a measurement type?
- What is the difference between area and volume?

Assistant retrieves information from existing documentation and responds conversationally.

Assistant should not simply paste documentation.

Assistant should summarize and contextualize.

Application Awareness

Assistant receives live context from frontend.

Examples:

- Current page
- Current module
- Current workflow
- Current item selected
- Visible UI elements

Assistant can answer:

- Where am I?
 - What am I editing?
 - What does this field mean?
 - What should I do next?
-

Guided Workflow Mode

The current Copilot system contains valuable workflow logic.

That logic should be preserved and exposed to the assistant.

Assistant can access:

- Current workflow
- Current step
- Completion requirements
- Next step
- Target UI element

Assistant provides conversational guidance instead of rigid tutorial popups.

UI Highlighting

The assistant should be capable of requesting UI highlights.

Examples:

- Highlight button
- Highlight text field
- Highlight dropdown
- Highlight table row
- Highlight menu item

Assistant should return structured highlight instructions.

Example:

```
{ "action": "highlight", "target": "confirm-component-button" }
```

Frontend performs visual highlight.

Possible visual treatments:

- Pulsing outline
- Glow effect
- Arrow indicator
- Temporary spotlight effect

This functionality is highly desirable for onboarding and should be included in MVP architecture.

Floating Assistant Interface

Replace Copilot UI.

Requirements:

- Floating widget
- Draggable
- Collapsible
- Persistent across pages
- Chat history retained
- Streaming responses

Modes:

Default: Assistant

Future: Assistant + Voice

Future: Mobile Assistant

Design architecture so future modes can reuse the same backend assistant.

Context Architecture

Assistant should receive structured context.

Example:

```
{ "userId": "123", "accountId": "456", "currentPage": "components", "currentModule": "component_builder",  
  "currentWorkflow": "create_component", "currentStep": "measurement_type", "selectedItem": { "id": "789",  
  "name": "Wall Surface" }, "visibleElements": [ { "id": "measurement-dropdown", "label": "Measurement  
Type" } ] }
```

This context should be assembled by the application.

The AI should never attempt to infer application state.

Knowledge Sources

The assistant should have access to:

Source 1

Help Documentation

Current help system.

Indexed for semantic search.

Source 2

Workflow Definitions

Current Copilot flows.

Converted into structured workflow definitions.

Example:

```
{ "workflowId": "create_component", "steps": [ { "id": "select_measurement", "instruction": "Choose a measurement type", "targetElement": "measurement-dropdown", "completionEvent": "measurement_selected" } ] }
```

Source 3

Live Application Context

Current page.

Current item.

Current workflow state.

Visible UI.

User permissions.

Technical Stack

Recommended:

Frontend

- Existing application frontend
- Floating assistant component
- WebSocket or SSE response streaming

Backend

- Existing backend
- Assistant service layer
- Workflow service layer
- Context service layer

Storage

- PostgreSQL
- pgvector

AI

- OpenAI Responses API
- GPT-5 Mini

Reasoning:

Excellent quality.

Low cost.

Scales well.

Future models can be swapped without architectural changes.

Assistant Tools

The assistant should not access the database directly.

Expose application tools/functions.

Initial tools:

`search_help_docs()`

`get_current_context()`

`get_current_workflow()`

`get_current_step()`

`get_ui_element_details()`

request_ui_highlight()

Future tools:

get_schedule()

get_tasks()

get_notifications()

get_account_summary()

create_event_draft()

create_task_draft()

submit_user_action()

Security Model

MVP permissions:

Read-only.

Assistant can:

- Read documentation
- Read workflow state
- Read application context

Assistant cannot:

- Modify data
- Create records
- Delete records
- Schedule events

Future versions may support limited write permissions.

All write actions must:

- Require explicit user approval
- Be logged
- Be permission validated

Future Mobile Assistant Vision

This MVP must be designed as the foundation for:

Mobile Assistant

Capabilities:

- Text chat
- Voice chat
- Task management
- Schedule management
- Notifications
- Reminders

Assistant becomes the user's operational assistant.

Examples:

"What jobs do I have tomorrow?"

"Show me today's tasks."

"Create a reminder to call John at 3 PM."

"What quote was I working on?"

The architecture must support this future expansion without requiring a full rewrite.

Success Criteria

The assistant is considered successful when:

1. User can ask documentation questions naturally.
2. Assistant knows current page and context.
3. Assistant knows current workflow step.
4. Assistant can explain next actions.
5. Assistant can highlight UI elements.
6. Assistant can replace current Copilot experience.
7. Assistant architecture supports future mobile assistant development.

The goal is not to build a chatbot.

The goal is to build the first version of a persistent account-aware AI assistant platform.